

Lezione 4 - useEffect

Introduzione

Prima dell'introduzione degli Hooks, i componenti funzionali venivano usati quasi esclusivamente per ricevere props e restituire JSX.

Con l'arrivo dello **State Hook** ('useState') abbiamo potuto gestire dati dinamici nei componenti funzionali.

Tuttavia, molte operazioni non riguardano direttamente il rendering dell'interfaccia:

- recuperare dati da un backend
- iscriversi a flussi di eventi
- avviare timer o intervalli
- modificare il DOM o leggere informazioni esterne al componente

Queste operazioni sono chiamate **side effects**:

- **effetti collaterali che devono essere eseguiti in momenti specifici del ciclo di vita del componente.**

Per gestirli in modo corretto e prevedibile, React mette a disposizione l'**Effect Hook**, cioè `useEffect`.

Lo `useEffect`

Un effetto può essere eseguito in tre momenti fondamentali del ciclo di vita del componente:

1. **Quando il componente viene montato (render iniziale)**
2. **Quando stato o props cambiano, causando un nuovo render**
3. **Quando il componente viene smontato, se l'effetto prevede una cleanup function**

La frequenza con cui l'effetto viene eseguito dipende dalla dependency array, che vedremo più avanti.

Esempio pratico: Aggiornare il titolo della pagina

Questo esempio mostra tre concetti fondamentali:

- come usare `useState` per gestire un valore dinamico (`name`)
- come usare `useEffect` per reagire ai cambiamenti dello stato

- come gestire un input controllato in React



React JSX

```
1  export default function PageTitle() {
2
3    // Stato locale: contiene il nome digitato dall'utente.
4    const [name, setName] = useState('');
5
6    /*
7    =====
8    Effetto con useEffect
9    =====
10   La funzione passata a useEffect viene eseguita:
11   - dopo ogni render,
12   - ma solo quando cambia una delle dipendenze elencate
    nell'array.
13
14   In questo caso, aggiorniamo il titolo del browser ogni volta
    che cambia 'name'.
15   */
16   useEffect(() => {
17     document.title = `Hi, ${name}`;
18   }, [name]); // Esegui l'effetto solo quando 'name' cambia
19
20   return (
21     <div>
22       <p>Use the input below to rename this page!</p>
23
24       <input
25         type="text"
26
27         // Input controllato: il valore dell'input è
    sempre sincronizzato con lo stato
28         onChange={({ target }) => setName(target.value)}
29
30         value={name}
31       />
32     </div>
33   );
34 }
```

Spiegazione:

1. Lo stato locale:

`useState('')` crea una variabile di stato (`name`) e una funzione per aggiornarla (`setName`).

Ogni volta che `setName()` viene chiamato, React esegue un nuovo render del componente.

2. L'effetto (`useEffect`)

`useEffect(() => { ... }, [name])` definisce un effetto che:

- viene eseguito **dopo il rendering**
- aggiorna il titolo della scheda del browser usando il valore corrente di `name`
- si attiva **solo** quando cambia `name` , grazie alla *dependency array*

Comportamenti alternativi della dependency array:

- **senza array**: l'effetto viene eseguito *ad ogni render*
- **array vuota** `[]` : l'effetto viene eseguito *solo al montaggio* del componente

3. Input controllato:

L'input è sincronizzato con lo stato tramite:



React JSX

```
1 value={name}
2 onChange={({ target }) => setName(target.value)}
```

Questo implica che:

- il testo mostrato è sempre quello presente nello stato
- ogni modifica dell'utente aggiorna il valore dello stato tramite `setName`
- l'aggiornamento dello stato innesca un nuovo render
- e l'effetto si riattiva aggiornando il titolo della pagina

Questo è il classico comportamento dei *controlled components* in React.

Cosa sono i controlled components

In React, un **controlled component** è un elemento del form (come `<input>` , `<textarea>` , `<select>`) il cui **valore è interamente gestito dallo stato React**.

In altre parole:

Un componente è “controllato” quando **React** è l'unica fonte di verità del suo valore.

Come funziona un controlled component

Quando un input è controllato:

1. Il suo valore visibile proviene da una variabile di stato (`value={...}`)
2. Ogni modifica dell'utente genera un evento `onChange`
3. L'handler dell'evento aggiorna lo stato con il nuovo valore
4. Il nuovo stato provoca un re-render → e React aggiorna l'input

Quindi il flusso è:

<> UI → onChange → setState → render → aggiorna valore dell'input

Per capire meglio facciamo un esempio semplice



React JSX

```
1  const [name, setName] = useState("");
2  <input
3    value={name}
4    onChange={(e) => setName(e.target.value)}
5  />
```

Qui l'input mostra sempre il valore presente nello stato `name`.

Se provi a togliere `onChange`, l'input diventerebbe *a sola lettura*, perché React controlla tutto.

🔗 Perché usarli?

I controlled components permettono a React di:

- validare il contenuto in tempo reale
- formattare automaticamente i dati (es. uppercase)
- bloccare input non validi
- sincronizzare più campi tra loro
- mantenere uno stato affidabile e prevedibile durante tutto il ciclo di vita del form

In pratica, i form diventano **parte del flusso dichiarativo di React**, invece di “vivere di vita propria” nel DOM.

Gli effetti nei componenti funzionali

L'**Effect Hook** (`useEffect`) permette a un componente funzionale di **eseguire del codice ogni volta che viene renderizzato o ri-renderizzato**.

Si usa per gestire tutte quelle operazioni che producono *effetti collaterali* rispetto al semplice rendering, come:

- aggiornare il DOM manualmente
- sincronizzare dati esterni
- recuperare informazioni da un back-end
- ascoltare eventi del browser
- gestire timer o intervalli

In combinazione con lo stato, `useEffect` consente di introdurre **comportamenti dinamici** nella nostra applicazione.

Supponiamo, ad esempio, di voler aggiornare il **titolo della scheda del browser** ogni volta che l'utente digita nell'input.

Riprendiamo il componente usato come esempio:



React JSX

```
1  import React, { useState, useEffect } from 'react';
2
3  function PageTitle() {
4    const [name, setName] = useState('');
5
6    useEffect(() => {
7      document.title = `Hi, ${name}`;
8    });
9
10   return (
11     <div>
12       <p>Use the input field below to rename this page!</p>
13       <input
14         onChange={({ target }) => setName(target.value)}
15         value={name}
```

```

16         type='text'
17     />
18 </div>
19 );
20 }

```

Spiegazione

1. Import dello useEffect:



React JSX

```
1 import { useEffect } from 'react';
```

`useEffect` **non restituisce alcun valore e non viene utilizzato per ottenere dati dal componente:**

serve esclusivamente a registrare una funzione da eseguire dopo il render.

2. Passare la funzione di effetto:

la struttura base è:



React JSX

```

1 useEffect(() => {
2     // codice dell'effetto
3 });

```

Quindi nel nostro esempio:



React JSX

```

1 useEffect(() => {
2     document.title = `Hi, ${name}`;
3 });

```

Questa funzione viene eseguita **dopo ogni render del componente.**

Ogni volta che l'utente digita nell'input, si verifica un re-render e l'effetto aggiorna il titolo del documento.

3. Relazione tra stato ed effetto

L'handler `onChange` aggiorna lo stato tramite `setName(target.value)`.

Ogni aggiornamento dello stato provoca un nuovo render, e quindi:

- **React aggiorna il DOM**
- **Dopo l'aggiornamento del DOM, React esegue la funzione passata a `useEffect`**
- **L'effetto modifica il titolo della pagina usando il valore *corrente* di `name`**

È importante notare che, anche se l'effetto viene eseguito *dopo* il render, esso ha comunque accesso alle variabili presenti nello scope del componente (quindi allo stato aggiornato).

Esempio aggiuntivo: Incremento con effetto

In questo esempio l'effetto mostra un alert ogni volta che il valore dello stato cambia.



React JSX

```
1  const Counter = () => {
2    const [count, setCount] = useState(0);
3
4    useEffect(() => {
5      alert(`Il valore corrente del contatore è: ${count}`);
6    });
7
8    const handleClick = () => {
9      setCount(prevCount => prevCount + 1);
10   };
11
12   return (
13     <div>
14       <p>You clicked {count} times</p>
15       <button onClick={handleClick}>
16         Click me
17       </button>
18     </div>
19   );
20 }
```

Spiegazione:

- `useState(0)` crea uno **stato locale** `count` e una funzione **updater** `setCount`.

- Ogni pressione del pulsante **aggiorna lo stato** usando la forma funzionale dello state setter:



React JSX

```
1  setCount(prevCount => prevCount + 1);
```

- Poiché React esegue un **re-render** del componente ogni volta che lo stato cambia, l'**effect** definito con `useEffect` viene rieseguito, mostrando un alert con il **valore aggiornato di** `count`.

Questo esempio evidenzia come `useEffect` permetta di **rispondere in modo dichiarativo ai cambiamenti dello stato**, senza dover gestire manualmente il ciclo di aggiornamento del DOM.

Pulire gli effetti con `useEffect`

Alcuni effetti nei componenti React richiedono una **pulizia** quando non sono più necessari.

Un esempio tipico è l'aggiunta di **event listener** direttamente al DOM, oltre agli elementi JSX presenti nel componente.

⚠ È importante rimuovere questi listener quando non servono più, per **evitare perdite di memoria**, bug o degrado delle prestazioni.

Esempio pratico: gestione di un contatore di click sul documento



React JSX

```
1  import React, { useState, useEffect } from 'react';
2
3  const Count = () => {
4    // Stato locale: tiene traccia del numero di click sul
    documento
5    const [clickCount, setClickCount] = useState(0);
6
7    // Event handler: incrementa il contatore di 1
8    const increment = () => setClickCount(prevClickCount =>
    prevClickCount + 1);
9
10   useEffect(() => {
```

```

11      // Aggiunge un listener per l'evento 'mousedown' sull'oggetto
    document
12      document.addEventListener('mousedown', increment);
13
14      // Funzione di pulizia: rimuove il listener quando il
    componente viene re-renderizzato o smontato
15      return () => {
16          document.removeEventListener('mousedown', increment);
17      };
18  });
19
20  return (
21      <h1>Document Clicks: {clickCount}</h1>
22  );
23  };

```

Spiegazione passo-passo:

1. Stato locale

- `useState(0)` crea lo stato `clickCount` e la funzione updater `setClickCount`.
- Ogni click sul documento aggiorna lo stato tramite la forma **funzionale**: `prevClickCount => prevClickCount + 1`.
- Questo assicura che ogni aggiornamento parta dal **valore precedente** corretto.

2. Effetto con `useEffect`

- L'hook `useEffect` viene chiamato **dopo ogni render**.
- Aggiunge un listener `mousedown` al `document` che richiama `increment`.
- Ogni pressione del mouse sul documento triggera la funzione `increment`, aggiornando lo stato e causando un re-render.

3. Funzione di pulizia

- React chiama automaticamente la funzione di pulizia **prima di ogni nuovo render** o quando il componente viene smontato.
- La funzione di pulizia rimuove il listener:



React JSX

```

1  document.removeEventListener('mousedown', increment)

```

- Senza questa funzione, ad ogni render verrebbe aggiunto un nuovo listener, causando:
 - **incrementi multipli per singolo click**
 - **comportamento apparentemente “randomico” del contatore**
 - **potenziali perdite di memoria**

4. Dettagli sull'evento `mousedown`

- Si attiva **quando il pulsante del mouse viene premuto**, non quando viene rilasciato.
- Se vuoi rilevare il rilascio del pulsante, usa invece `mouseup`.
- L'evento è utile per azioni immediate al click, come drag & drop o disegno su canvas.
- Viene seguito da un eventuale `click` se il pulsante viene rilasciato sullo stesso elemento.

☰ In sintesi

- Gli effetti in React **possono avere conseguenze collaterali** sul DOM.
- Quando creano risorse esterne, come **event listener**, è necessario fornire una **funzione di pulizia**.
- La funzione di pulizia evita **duplicazioni di listener, bug e problemi di prestazioni**.
- In questo esempio, ogni click sul documento incrementa **sempre correttamente** il contatore, grazie alla combinazione di:
 - **useState funzionale**
 - **useEffect con funzione di cleanup**

Controllare quando vengono richiamati gli effetti

La funzione `useEffect()` **esegue il suo primo argomento, cioè la funzione di effetto, dopo ogni rendering di un componente.**

Finora abbiamo visto come restituire una **funzione di pulizia** per evitare bug e problemi di prestazioni, ma a volte vogliamo **saltare del tutto l'esecuzione dell'effetto sui re-render** successivi.

È comune, ad esempio, voler eseguire un effetto **solo al montaggio del componente** (cioè alla prima renderizzazione), ma **non ad ogni re-render**.

Il dependency array

L'Effect Hook ci permette di gestire questo comportamento in modo semplice:

- passando un **array vuoto** come secondo argomento di `useEffect()`.

Questo array è chiamato **dependency array** (matrice di dipendenze):

- **serve a dire a React quando eseguire l'effetto e quando ignorarlo.**
- **Se l'array è vuoto `[]`, l'effetto viene eseguito solo una volta, al montaggio del componente.**
- **Se l'array contiene variabili di stato o props, l'effetto viene rieseguito solo quando una di queste cambia.**

In altre parole, l'array di dipendenze permette di **controllare con precisione il comportamento dell'effetto**.

Esempio: effetto eseguito al primo render e pulizia al dismount



React JSX

```
1  useEffect(() => {
2    alert("Component rendered for the first time");
3
4    return () => {
5      alert("Component is being removed from the DOM");
6    };
7  }, []);
```

Senza l'array vuoto `[]`:

- i messaggi di alert verrebbero mostrati **ad ogni render**, cosa indesiderata. **Passando `[]`, invece:**
- **il messaggio di “primo render” appare solo all’inizio, e la funzione di pulizia viene richiamata solo quando il componente viene smontato.**

Esempio pratico: un timer con `useEffect`



React JSX

```
1  import React, { useState, useEffect } from 'react';
2
3  const Timer = () => {
```

```

4      // Stato per il timer
5      const [time, setTime] = useState(0);
6
7      // Stato per un input controllato
8      const [name, setName] = useState("");
9
10     // Event handler compatto per aggiornare lo stato dell'input
11     const handleChange = ({ target }) => setName(target.value);
12
13     // Effetto: avvia un timer che incrementa il contatore ogni
    secondo
14     useEffect(() => {
15         const intervalId = setInterval(() => {
16             // Forma funzionale dello state setter: garantisce
    che l'ultimo valore venga sempre aggiornato
17             setTime(prevTime => prevTime + 1);
18         }, 1000);
19
20         // Pulizia dell'effetto: rimuove il timer quando il
    componente viene smontato
21         return () => {
22             clearInterval(intervalId);
23         };
24     }, []); // Array di dipendenze vuoto: effetto eseguito solo
    al mount
25
26     return (
27         <>
28             <h1>Time: {time}</h1>
29             <input
30                 type="text"
31                 value={name}           // Controlled component
32                 onChange={handleChange}
33             />
34         </>
35     );
36 };

```

Spiegazione passo-passo

1. Stati locali

- `time` **tiene traccia dei secondi trascorsi.**
- `name` **gestisce il valore dell'input controllato.**

2. Effetto del timer

- `useEffect` **crea un intervallo con** `setInterval` **che incrementa** `time` **di 1 ogni secondo.**
- **Grazie alla forma funzionale** `setTime(prevTime => prevTime + 1)`, **il timer utilizza sempre l'ultimo valore dello stato, evitando problemi di "stale state".**

3. Funzione di pulizia

- `return () => clearInterval(intervalId)` **rimuove il timer quando il componente viene smontato.**
- **Senza la pulizia, ogni re-render aggiungerebbe un nuovo intervallo, causando incrementi multipli e potenziali problemi di prestazioni.**

4. Array di dipendenze vuoto

- Garantisce che il timer venga creato **solo una volta**, al montaggio.
- **La funzione di pulizia viene eseguita solo al dismount, evitando duplicazioni di intervalli.**

5. Input controllato

- Il valore dell'input è sempre sincronizzato con lo stato `name`.
- Ogni modifica aggiorna lo stato, causando un re-render, ma non riavvia il timer grazie all'array di dipendenze vuoto.

☰ In sintesi

Con `useEffect` e l'array di dipendenze possiamo **controllare con precisione quando eseguire effetti e gestire correttamente le risorse** come timer o listener, evitando bug e perdite di memoria.

Fetch Data con `useEffect`

Uno dei casi d'uso più comuni dell'**Effect Hook** (`useEffect`) in React è il **recupero dei dati da un server** (fetch dei dati).

Quando un componente React deve ottenere dati esterni (API REST, backend, mock server, ecc.), è fondamentale **controllare con precisione quando l'effetto viene eseguito**.

Questo perché una chiamata al server è un'operazione costosa e non dovrebbe essere eseguita più volte del necessario.

Perché è importante controllare il fetch dei dati

Ogni chiamata non necessaria al server comporta un costo in termini di:

- elaborazione
- prestazioni
- consumo di dati (soprattutto su dispositivi mobili)
- eventuali costi economici nel caso di API a pagamento

Per questo motivo, **non è quasi mai corretto eseguire un fetch dopo ogni render del componente.**

Comportamento di default di useEffect nel fetch

Per impostazione predefinita, `useEffect` si comporta così:



React JSX

```
1  useEffect(() => {  
2    // fetch dei dati  
3  });
```

In questa forma, l'effetto viene eseguito **dopo ogni render** del componente.

Questo è un problema perché:

- ogni aggiornamento di stato (ad esempio la digitazione in un input, un click, una modifica locale)
- provoca un nuovo render
- che a sua volta rilancia l'effetto
- e quindi una nuova chiamata al server, anche se i dati non sono cambiati

Fetch eseguito solo al primo render (mount)

Se i dati **non dipendono da alcuna variabile**, possiamo passare un **dependency array vuoto** come secondo argomento di `useEffect` :



React JSX

```
1  useEffect(() => {  
2    // fetch dei dati  
3  }, []);
```

In questo modo:

- il fetch viene eseguito **una sola volta**
- subito dopo il primo render del componente (fase di mount)
- i dati vengono poi riutilizzati nei render successivi

Questo approccio è tipico per:

- dati iniziali
- configurazioni
- contenuti statici recuperati dal server

Pattern fondamentale: **useEffect + useState**

Nel recupero dei dati, `useEffect` viene quasi sempre utilizzato insieme a `useState`.

Il pattern generale è il seguente:



React JSX

```
1  const [data, setData] = useState();
2
3  useEffect(() => {
4    fetch(...)
5      .then(response => response.json())
6      .then(result => setData(result));
7  }, []);
```

In questo pattern:

- `useEffect` si occupa del fetch dei dati
- `setData` salva i dati nello stato locale del componente
- React esegue un nuovo render usando i dati appena recuperati
- il fetch **non viene ripetuto inutilmente**

Questo è uno dei pattern più importanti in React per la gestione dei dati asincroni.

Fetch con dependency array non vuoto

Quando i dati **dipendono da una variabile di stato**, quella variabile deve essere inserita nel dependency array.

Esempio:



React JSX

```
1  const [forecastType, setForecastType] = useState('/daily');
2
3  useEffect(() => {
4    get(forecastType).then(response => {
5      setData(response.data);
6    });
7  }, [forecastType]);
8
```

In questo caso:

- l'effetto viene eseguito dopo il primo render
- viene rieseguito **solo quando** `forecastType` **cambia**
- se `forecastType` rimane invariato, il fetch viene saltato

In questo modo si ottiene un fetch **reattivo ma controllato**.

Caso di studio: Forecast component

Di seguito un esempio completo di componente che utilizza:

- `useState` per gestire i dati
- `useEffect` per il fetch
- un dependency array per controllare quando richiamare il server



React JSX

```
1  import { useState, useEffect } from "react";
2  import { get } from "../mockBackend/fetch";
3
4  const Forecast = () => {
5    const [data, setData] = useState();
6    const [notes, setNotes] = useState({});
7    const [forecastType, setForecastType] = useState("/daily");
8
9    useEffect(() => {
10      alert("Requested data from server...");
11
12      get(forecastType).then((response) => {
13        alert("Response: " + JSON.stringify(response, "", 2));
14        setData(response.data);
15      });
16    }, [forecastType]);

```

```

17
18   const handleChange = (itemId) => ({ target }) =>
19     setNotes((prev) => ({
20       ...prev,
21       [itemId]: target.value,
22     }));
23
24   // Gestione dello stato di loading
25   if (!data) {
26     return <p>Loading...</p>;
27   }
28
29   return (
30     <div className="App">
31       <h1>My Weather Planner</h1>
32
33       <div>
34         <button onClick={() => setForecastType("/daily")}>5-
day</button>
35         <button onClick={() =>
setForecastType("/hourly")}>Today</button>
36       </div>
37
38       <table>
39         <thead>
40           <tr>
41             <th>Summary</th>
42             <th>Avg Temp</th>
43             <th>Precip</th>
44             <th>Notes</th>
45           </tr>
46         </thead>
47         <tbody>
48           {data.map((item) => (
49             <tr key={item.id}>
50               <td>{item.summary}</td>
51               <td>{item.temp.avg}°F</td>
52               <td>{item.precip}%</td>
53               <td>
54                 <input
55                   value={notes[item.id] || ""}
56                   onChange={handleChange(item.id)}
57                 />
58               </td>

```

```

59         </tr>
60     }}
61 </tbody>
62 </table>
63 </div>
64 );
65 };
66
67 export default Forecast;
68

```

Import delle dipendenze



React JSX

```

1  import { useState, useEffect } from "react";
2  import { get } from "../mockBackend/fetch";

```

- `useState` : Hook che consente di gestire lo **stato locale** del componente.
- `useEffect` : Hook che consente di eseguire **effetti collaterali** dopo il render (in questo caso il fetch dei dati).
- `get` : funzione che simula una chiamata HTTP a un backend (mock), restituendo una Promise con i dati meteo.

Dichiarazione del componente



React JSX

```

1  const Forecast = () => {

```

`Forecast` è un **functional component** React.

Tutta la logica di stato, effetti e rendering è contenuta all'interno di questa funzione.

Dichiarazione degli state



React JSX

```

1  const [data, setData] = useState();
2  const [notes, setNotes] = useState({});
3  const [forecastType, setForecastType] = useState("/daily");

```

1. data

- Contiene i **dati restituiti dal server**.
- Inizialmente è `undefined` perché il fetch è asincrono.
- Viene valorizzato dopo la risposta del backend tramite `setData`.

2. notes

- Oggetto che memorizza le note inserite dall'utente.
- La chiave è l' `id` dell'elemento della previsione.
- Il valore è il testo digitato nell'input corrispondente.
- È uno stato **locale e indipendente** dai dati del server.

3. forecastType

- Determina quale endpoint deve essere interrogato (`/daily` o `/hourly`).
- Influenza direttamente il comportamento del fetch.
- È inserito nel dependency array di `useEffect`.

useEffect per il fetch dei dati



React JSX

```
1  useEffect(() => {
2    alert("Requested data from server...");
3
4    get(forecastType).then((response) => {
5      alert("Response: " + JSON.stringify(response, "", 2));
6      setData(response.data);
7    });
8  }, [forecastType]);
```

Cosa fa questo effetto

- Viene eseguito **dopo il primo render** del componente.
- Viene rieseguito **solo quando cambia** `forecastType`.
- Effettua una chiamata al backend simulato tramite `get(forecastType)`.

Flusso di esecuzione

1. Il componente viene renderizzato.
2. `useEffect` viene eseguito.

- Viene richiesta la risorsa `/daily` o `/hourly`.
- Alla risoluzione della Promise:
 - i dati vengono salvati nello stato tramite `setData`
 - React esegue un nuovo render con i dati aggiornati.

Grazie al dependency array `[forecastType]`, i **render causati da input o note non rilanciano il fetch**.

Gestione dell'input dinamico (notes)



React JSX

```
1  const handleChange = (itemId) => ({ target }) =>
2    setNotes((prev) => ({
3      ...prev,
4      [itemId]: target.value,
5    }));
```

Questa funzione:

- è una **higher-order function** (ritorna una funzione)
- riceve l'`id` dell'elemento corrente
- aggiorna lo stato `notes` in modo immutabile

Cosa succede internamente

- `prev` rappresenta lo il valore attuale di `notes`
- lo spread operator (`...prev`) evita la perdita dei valori già inseriti
- `[itemId]` permette di aggiornare dinamicamente la chiave corretta

Ogni input della tabella ha così il **proprio valore indipendente**.

Gestione dello stato di loading



React JSX

```
1  if (!data) {
2    return <p>Loading...</p>;
3  }
```

Poiché il fetch è asincrono:

- al primo render `data` è `undefined`

- senza questo controllo, `data.map(...)` genererebbe un errore

Questo blocco:

- interrompe il rendering principale
- mostra un messaggio di caricamento
- ripristina il rendering normale quando `data` è disponibile

Rendering del JSX



React JSX

```
1  return (  
2    <div className="App"></div>
```

Titolo dell'applicazione

```
<h1>My Weather Planner</h1>
```

Elemento puramente presentazionale.

Bottoni per cambiare il tipo di previsione



React JSX

```
1  <button onClick={() => setForecastType("/daily")}>5-day</button>  
2  <button onClick={() => setForecastType("/hourly")}>Today</button>
```

- Ogni bottone aggiorna lo stato `forecastType`.
- Il cambio di stato provoca:
 1. un nuovo render
 2. la riesecuzione di `useEffect`
 3. una nuova chiamata al server

Questo è un esempio di **fetch reattivo controllato**.

Rendering della tabella dei dati



React JSX

```
1  {data.map((item) => (  
2    <tr key={item.id}>
```

- `map` itera l'array dei dati ricevuti dal backend.
- `key={item.id}` è necessaria per il corretto funzionamento del Virtual DOM.

Celle della tabella



React JSX

```
1 <td>{item.summary}</td>
2 <td>{item.temp.avg}°F</td>
3 <td>{item.precip}%</td>
4
```

Visualizzano i dati restituiti dal server.

Input controllato per le note



React JSX

```
1 <input
2   value={notes[item.id] || ""}
3   onChange={handleChange(item.id)}
4 />
5
```

- È un **controlled component**:
 - il valore è sempre sincronizzato con lo stato React
- Ogni input è legato alla propria voce tramite `item.id`
- La digitazione provoca:
 - aggiornamento dello stato `notes`
 - re-render del componente
 - **nessuna chiamata al server**

Export del componente

```
export default Forecast;
```

Rende il componente disponibile per l'importazione in altri file.

☰ Conclusione

L'uso corretto del dependency array nel fetch dei dati permette di:

- evitare chiamate ridondanti

- migliorare le prestazioni
- sincronizzare lo stato con i dati remoti
- rendere il componente prevedibile e manutenibile

In React, **fetch + useEffect + useState** costituiscono un pattern fondamentale per la gestione dei dati asincroni.

Regole fondamentali degli hooks

Quando si utilizzano gli Hooks in React, è essenziale rispettare **due regole fondamentali**:

1. **Gli Hooks devono essere chiamati solo al livello superiore del componente**
2. **Gli Hooks possono essere chiamati solo all'interno di funzioni React**

Durante i primi esercizi con `useState` e `useEffect`, queste regole risultano spesso intuitive da rispettare.

Tuttavia, man mano che la logica dei componenti diventa più complessa e il numero di Hooks aumenta, è facile incorrere in errori se non si ha ben chiaro il motivo di tali vincoli.

Perché React impone queste regole

Quando React costruisce e aggiorna il **Virtual DOM**, la libreria **invoca ripetutamente le funzioni che definiscono i componenti** in risposta alle interazioni dell'utente.

React non identifica gli Hooks tramite il loro nome, ma in base all'ordine con cui vengono chiamati all'interno del componente.

Questo significa che React associa ogni chiamata a `useState`, `useEffect`, ecc. a una precisa posizione nella funzione.

Per questo motivo:

- l'ordine delle chiamate agli Hooks **deve essere sempre lo stesso**
- React deve poter "ritrovare" ogni Hook nello stesso punto a ogni render

Se un Hook viene chiamato solo in certe condizioni (ad esempio dentro un `if` o un ciclo), l'ordine delle chiamate cambia tra un render e l'altro, causando comportamenti imprevedibili o errori.

Cosa non fare: Hooks dentro condizioni

Un esempio di codice **non valido** è il seguente:



React JSX

```
1  if (userName !== '') {  
2    useEffect(() => {  
3      localStorage.setItem('savedUserName', userName);  
4    });  
5  }
```

In questo caso, `useEffect` viene chiamato **solo se la condizione è vera**, rompendo la regola dell'ordine costante degli Hooks.

Approccio corretto

Lo stesso risultato può essere ottenuto **chiamando sempre l'Hook**, ma spostando la logica condizionale *all'interno* dell'effetto:



React JSX

```
1  useEffect(() => {  
2    if (userName !== '') {  
3      localStorage.setItem('savedUserName', userName);  
4    }  
5  });
```

In questo modo:

- l'Hook viene eseguito sempre nello stesso punto
- la logica condizionale non interferisce con il meccanismo interno di React

Seconda regola: dove è possibile usare gli Hooks

Gli Hooks possono essere utilizzati **solo**:

- **all'interno di componenti funzionali React**
- **all'interno di Custom Hooks**

Non è possibile utilizzare `useState`, `useEffect`, ecc. in:

- funzioni JavaScript normali
- classi

- callback esterne ai componenti

I **Custom Hooks** sono funzioni che iniziano con `use` e permettono di **incapsulare e riutilizzare logica stateful** tra più componenti, mantenendo il rispetto delle regole degli Hooks.

Esempio del componente Shop

Per comprendere meglio questi concetti prendiamo ad esempio il componente Shop :



React JSX

```
1  const Shop = () => {
2
3      const [categories, setCategories] = useState();
4
5      if (categories) {
6
7          const [selectedCategory, setSelectedCategory] =
useState();
8
9          const [items, setItems] = useState({});
10
11     }
12
13     if (!categories) {
14
15         useEffect(() => {
16
17             get('/categories').then((response) => {
18
19                 setCategories(response.data);
20
21             });
22
23         });
24
25     }
26
27
28
29     if (selectedCategory && !items[selectedCategory]) {
30         useEffect(() => {
```

```
31
32     get(`/items?
category=${selectedCategory}`).then((response) => {
33
34         setItems((prev) => ({ ...prev, [selectedCategory]:
response.data }));
35
36     });
37
38     });
39
40     }
41
42
43
44     if (!categories) {
45
46         return <p>Loading..</p>;
47
48     }
49
50
51
52     return (
53
54         <div className='App'>
55
56             <h1>Clothes 'n Things</h1>
57
58             <nav>
59
60                 {categories.map((category) => (
61
62                     <button key={category} onClick={() =>
setSelectedCategory(category)}>
63
64                         {category}
65
66                     </button>
67
68                 ))}
69
70             </nav>
71
```

```

72     <h2>{selectedCategory}</h2>
73
74     <ul>
75
76         {!items[selectedCategory]
77
78             ? null
79
80             : items[selectedCategory].map((item) => <li key={item}>
{item}</li>)}
81
82     </ul>
83
84 </div>
85
86 );
87
88 }

```

Come possiamo notare il componente `Shop` presenta diversi problemi strutturali legati all'uso scorretto degli Hooks.

Errore principale: Hooks annidati in condizioni



React JSX

```

1  if (categories) {
2    const [selectedCategory, setSelectedCategory] = useState();
3    const [items, setItems] = useState({});
4  }

```

Qui:

- due `useState` sono definiti **dentro uno statement condizionale**
- questo viola direttamente la prima regola degli Hooks

Lo stesso problema si ripresenta con `useEffect`, anch'esso inserito dentro un `if`.

Il risultato è un componente che:

- può funzionare apparentemente in alcuni casi
- ma è intrinsecamente **instabile e non conforme al modello React**

Refactoring del componente Shop

Il refactoring ha come obiettivo principale **ripristinare la correttezza strutturale** del componente, rispettando le regole degli Hooks.

1. Spostare tutti gli Hooks al livello superiore



React JSX

```
1  const [categories, setCategories] = useState(null);
2  const [selectedCategory, setSelectedCategory] = useState(null);
3  const [items, setItems] = useState({});
```

Tutti gli Hooks vengono:

- dichiarati all'inizio del componente
- inizializzati in modo esplicito (`null` o `{}`)

Questo rende il codice più leggibile e prevedibile.

2. Fetch delle categorie: uso corretto di `useEffect`

L'intenzione originale era eseguire il fetch delle categorie **una sola volta**.

Il modo corretto per ottenere questo comportamento è utilizzare un `useEffect` con **array di dipendenze vuoto**:



React JSX

```
1  useEffect(() => {
2    get('/categories').then((response) => {
3      setCategories(response.data);
4    });
5  }, []);
```

In questo modo:

- l'effetto viene eseguito solo dopo il primo render
- non è necessario alcun `if (!categories)`
- il comportamento è stabile e conforme alle regole

3. Fetch degli item in base alla categoria selezionata

Il secondo `useEffect` gestisce il caricamento degli articoli quando l'utente seleziona una categoria:



```
1  useEffect(() => {
2    if (selectedCategory && !items[selectedCategory]) {
3      get(`/items?category=${selectedCategory}`).then((response) =>
4        {
5          setItems((prev) => ({
6            ...prev,
7            [selectedCategory]: response.data,
8          }));
9        }
10   }, [selectedCategory, items]);
```

Qui la logica è corretta perché:

- l'Hook è sempre chiamato
- la condizione è interna all'effetto
- il fetch viene eseguito solo se i dati non sono già presenti

Questo approccio evita chiamate ridondanti al backend.

Hooks separati per effetti separati

Nella sezione precedente abbiamo visto come **gli Hooks debbano essere chiamati sempre al livello superiore del componente** e come la logica condizionale debba essere spostata *all'interno* degli effetti.

Un altro principio altrettanto importante riguarda **come organizzare lo stato e gli effetti** quando un componente gestisce più responsabilità.

Raggruppare o separare i dati nello stato

Come già accennato parlando di `useState`, quando più valori:

- **sono strettamente correlati**
- **cambiano insieme**
- **rappresentano un'unica entità logica**

può essere sensato **raggrupparli in una struttura unica**, come un oggetto o un array.

Tuttavia, questo approccio non è sempre vantaggioso.

Raggruppare dati **non correlati** in un unico stato può:

- aumentare la complessità degli aggiornamenti
- rendere più difficile capire *perché* uno stato cambia
- creare dipendenze indirette tra parti di codice indipendenti

Per questo motivo, una buona pratica in React è **separare le responsabilità, utilizzando Hook differenti per dati ed effetti differenti.**

Confronto tra due approcci

Primo caso: stato ed effetti accorpati

Nel seguente esempio, dati e comportamenti molto diversi vengono gestiti insieme:



React JSX

```
1  // Handle both position and menuItems with one useEffect hook.
2  const [data, setData] = useState({ position: { x: 0, y: 0 } });
3
4  useEffect(() => {
5    get('/menu').then((response) => {
6      setData((prev) => ({ ...prev, menuItems: response.data }));
7    });
8
9    const handleMove = (event) =>
10      setData((prev) => ({
11        ...prev,
12        position: { x: event.clientX, y: event.clientY }
13      }));
14
15    window.addEventListener('mousemove', handleMove);
16    return () => window.removeEventListener('mousemove',
17      handleMove);
18  }, []);
```

In questo caso:

- lo **stato** contiene sia `position` che `menuItems`
- un singolo `useEffect` gestisce:
 - una richiesta HTTP
 - un event listener sul mouse

- ogni aggiornamento richiede l'uso dello spread operator per preservare parti di stato non coinvolte

Il codice funziona, ma:

- è più difficile da leggere
- è meno modulare
- è più complesso da estendere o testare

Secondo caso: responsabilità separate

Separando stato ed effetti, il codice diventa più chiaro:



React JSX

```
1  // Gestisce menuItems con un solo hook useEffect.
2  const [menuItems, setMenuItems] = useState(null);
3  useEffect(() => {
4    get('/menu').then((response) => setMenuItems(response.data));
5  }, []);
6
7  // Gestisce position con un hook useEffect separato.
8  const [position, setPosition] = useState({ x: 0, y: 0 });
9  useEffect(() => {
10     const handleMove = (event) =>
11       setPosition({ x: event.clientX, y: event.clientY });
12
13     window.addEventListener('mousemove', handleMove);
14     return () => window.removeEventListener('mousemove',
15       handleMove);
16   }, []);
```

Qui ogni Hook:

- ha **una responsabilità ben definita**
- è più facile da comprendere isolatamente
- può essere riutilizzato o estratto in un Custom Hook

Non esiste una regola assoluta su quando raggruppare o separare i dati, ma con la pratica diventa più naturale riconoscere la soluzione più chiara e manutenibile.

Caso di studio: SocialNetwork

Nel componente `SocialNetwork` troviamo un esempio tipico di codice che **funziona**, ma che può essere migliorato dal punto di vista strutturale.

Versione iniziale



React JSX

```
1  const [data, setData] = useState(null);
2
3  useEffect(() => {
4    Promise.all([get('/menu'), get('/news-feed'),
5      get('/friends')]).then(
6      ([menuResponse, newsFeedResponse, friendsResponse]) => {
7        setData({
8          menu: menuResponse.data,
9          newsFeed: newsFeedResponse.data,
10         friends: friendsResponse.data
11       });
12     });
13   }, []);
```

In questo caso:

- più richieste di rete vengono gestite in **un unico effetto**
- dati concettualmente distinti (`menu` , `newsFeed` , `friends`) sono accorpati in un solo stato
- ogni modifica futura richiederà di intervenire su una struttura complessa

Anche se il codice è corretto, **la leggibilità e la manutenibilità possono migliorare**.

Refactoring: separare stato ed effetti

1. Stati distinti per dati distinti



React JSX

```
1  const [menu, setMenu] = useState(null);
2  const [newsFeed, setNewsFeed] = useState(null);
3  const [friends, setFriends] = useState(null);
```

Ogni variabile di stato ora rappresenta **un singolo dominio di dati**, rendendo immediatamente chiaro il suo scopo.

2. Effetti separati per ogni fetch



React JSX

```
1  useEffect(() => {
2    get('/menu').then((response) => {
3      setMenu(response.data);
4    });
5  }, []);
```



React JSX

```
1  useEffect(() => {
2    get('/news-feed').then((response) => {
3      setNewsFeed(response.data);
4    });
5  }, []);
```



React JSX

```
1  useEffect(() => {
2    get('/friends').then((response) => {
3      setFriends(response.data);
4    });
5  }, []);
```

Ogni `useEffect` :

- è responsabile di **una sola richiesta**
- viene eseguito una sola volta grazie al dependency array vuoto
- rispetta pienamente le regole degli Hooks